

WEG-Abrechnungsanwendung ist eine Flask Anwendung

Diese Flask-Web-Anwendung ist für die Verwaltung und Abrechnung einer Wohneigentümergeinschaft (WEG) entwickelt worden mit die folgende Hauptfunktionen bietet:

1. Transaktionsverwaltung: Import von Banktransaktionen aus CSV/Excel, manuelle Erfassung und Bearbeitung
2. Miteigentümergeverwaltung: Verwaltung der Miteigentümer mit ihren verschiedenen Anteilen (MEA, VF-Einheiten, TG-Einheiten)
3. Abrechnungssystem: Erstellung von Abrechnungen mit korrekter Kostenverteilung nach Verteilungsschlüsseln
4. Wirtschaftsplanverwaltung: Erstellung eines Wirtschaftsplan mit korrekter Kostenverteilung nach Verteilungsschlüsseln
5. Benutzerverwaltung von mehreren Benutzer (Rollen: Admin + Benutzer)
6. Datenexport von Transaktionen, Kontostand, Wirtschaftsplan und DB-Backup (letztes fehlt noch)
7. PDF-Export: Erstellung detaillierter PDF-Abrechnungen pro Miteigentümer Besonderheiten der Implementierung
 - Drei verschiedene Verteilungsschlüssel:
 - ** Einheiten: Alle Miteigentümer zahlen anteilig
 - ** VF-Einheiten: Nur Miteigentümer mit Wohnungen (VF=1) zahlen
 - ** TG-Einheiten: Nur Miteigentümer mit Tiefgaragenplätzen (TG=1) zahlen
 - Unterscheidung zwischen umlagefähigen und nicht umlagefähigen Kosten
 - Benutzeroberfläche mit Bootstrap und responsivem Design
 - PDF-Export der Abrechnungen mit ReportLab

Behobene Technische Probleme

1. Python-Versionskompatibilität (besonders für pandas)
2. Typfehler bei Decimal/Float-Berechnungen
3. Rendering-Probleme im Chrome-Browser
4. Division-by-Zero-Fehler in der Benutzeroberfläche
5. PDF-Formatierungsfehler

Erstelle Dateien

/weg-app/

|

|— app.py # Hauptapp

|— config.py # Konfigurationseinstellungen

|— models.py # Datenbankmodelle

- | — extensions.py # Limiter
- | — init_db.py # Initialisation der Basis-Datenbank
- |
- | — /manual-scripts/ # Verzeichnis für manuelle Skripts
- | | — backup.sh # Shell-Script für Backuperstellungen
- | | — init_admin.py # Initialisation von Administrator
- | | — init_kontostand.py # Initialisation von Kontostand
- | | — init_mappings.py # Initialisation von Mappings
- | | — migrate_db.py # DB Migration weil Flask Migrate fehlt
- | | — bankdaten_zu_csv.py # Skript zur Konvertierung einer Liste zu einer CSV
- |
- | — /routes/ # Verzeichnis für die Routen-Module
- | | — **init.py** # Macht das Verzeichnis zu einem Python-Paket
- | | — auth.py # Login, Logout, Benutzerfunktionen
- | | — dashboard.py # Dashboard und allgemeine Seiten
- | | — transaktionen.py # Transaktionsverwaltung
- | | — miteigentuemmer.py # Miteigentümmerverwaltung
- | | — abrechnung.py # Abrechnungsfunktionen
- | | — settings.py # Einstellungsverwaltung
- | | — wirtschaftsplan.py # Wirtschaftsplanfunktionen
- |
- | — /services/ # Neues Verzeichnis für die Geschäftslogik
- | | — **init.py** # Macht das Verzeichnis zu einem Python-Paket
- | | — transaktion_service.py # Geschäftslogik für Transaktionen
- | | — abrechnung_service.py # Geschäftslogik für Abrechnungen
- | | — anhang_service.py # Geschäftslogik um eine Rechnung an der Transaktion anzuhängen
- | | — kategorie_mapping_service.py # Geschäftslogik für Mapping von Kostenart und Kategorie zwischen Transaktionen und Wirtschaftsplan
- | | — kontostand_service.py # Geschäftslogik für Kontostand (manuelle Eingabe bis dato)
- | | — wirtschaftsplan_service.py # Geschäftslogik für Wirtschaftspläne
- |
- | — /static/
- | | — /css/
- | | | — bootstrap.min.css
- | | | — styles.css
- | | | — ...
- | | — /js/
- | | | — bootstrap.bundle.min.js
- | | | — jquery.min.js
- | | | — csrf-setup.js
- | | | — ...
- | | — /img/
- | | | — favicon.ico
- | | | — ...
- |

| — /templates/ # Templates in Unterordner organisieren

| | — layout.html # Haupt-Layout

| | — /auth/

| | | — login.html

| | | — profile.html

| | | — reset_request.html

| | | — reset_password.html

| | | — ...

| | — /dashboard/

| | | — dashboard.html

| | | — ...

| | — /emails/

| | | — password_reset.html

| | | — password_reset.txt

| | | — email_verification.html

| | | — email_verification.txt

| | | — transaction_notification.html

| | | — transaction_notification.txt

| | | — user_notification.html

| | | — user_notification.txt

| | | — wirtschaftsplan_notification.html

| | | — wirtschaftsplan_notification.txt

| | | — ...

| | — /errors/

| | | — 404.html

| | | — 500.html

| | | — simple_404.html

| | | — ...

| | — /transaktionen/

| | | — anhang.html

| | | — liste.html

| | | — neu.html

| | | — bearbeiten.html

| | | — upload.html

| | | — ...

| | — /settings/

| | | — data_management.html

| | | — export.html

| | | — index.html

| | | — kontostand_manage.html

| | | — kategorie_mapping.html

| | | — systeminfo.html

| | | — user_edit.html

| | | — user_new.html

| | | — users_list.html

```

|| └─ ...
| └─ /miteigentuemer/
|| └─ liste.html
|| └─ neu.html
|| └─ bearbeiten.html
|| └─ ...
| └─ /abrechnung/
|| └─ uebersicht.html
|| └─ detail.html
|| └─ ...
| └─ /wirtschaftsplan/
| └─ uebersicht.html
| └─ import.html
| └─ bearbeiten.html
| └─ neu.html
| └─ ...
|
| └─ /utils/ # Neues Verzeichnis für Hilfsfunktionen
| └─ /export/
|| └─ init.py # Macht das Verzeichnis zu einem Python-Paket
|| └─ export.py # Exportfunktionen für Transaktionen und Wirtschaftsplan
|| └─ ...
| └─ init.py # Macht das Verzeichnis zu einem Python-Paket
| └─ email_util.py # Hilffunktion für send_transaktion_notofication
| └─ email.py # send email und send passwort_reset
| └─ security.py # Prüfung ob ein strong password
| └─ utils.py # Allgemeine Hilfsfunktionen
|
| └─ /uploads/
|
| └─ requirements.txt
| └─ README.md
| └─ wsgi.py

```

Roadmap:

1. Weiterentwicklung der Anwendung:

- Vergleich von Wirtschaftsplan und tatsächlichen Kosten (Kategorien sind unterschiedlich) *check*
- Rechnungen (pdf/jpeg) an Transaktionen anhängen *check*
- Email-Versand für Änderungen bei Transaktionen/Wirtschaftsplan/Benutzer *check*
- Benutzerauthentifizierung verstärken *progressing*
- Mehrere Abrechnungszeiträume verwalten
- Verbesserung des PDF-Exports
- Kontostand online synchronisieren

2. Produktivdeploy:

- Konfiguration für sicheren Produktiveinsatz *check*
- Rate-Limiter *check*
- Content Security Policy *check but unsafe-mode*
- wsgi (waitress) verwenden *check*
- Secret Keys verstärken (config.py) *check*
- Backup-Strategien für die Datenbank
- Optimierungen für bessere Performance

3. Spezifische Anpassungen:

- Zusätzliche Verteilungsschlüssel
- Benutzerdefinierte Berichte